DSA Practical 8 Binary Search Tree

Name: Arjun Mankuskar

PRN:202401110038

Branch: CSE (AIML)

Batch: A2

CODE:-

```
#include <iostream>
using namespace std;

struct Node {
    int roll;
    Node* left;
    Node* right;

    Node(int r) {
        roll = r;
        left = right = NULL;
    }
};

class BST {
public:
    Node* root;

    BST() { root = NULL; }

    // Insert a new student roll number
```

```
Node* insert(Node* node, int roll) {
    if (node == NULL) return new Node(roll);

    if (roll < node->roll)
        node->left = insert(node->left, roll);
    else if (roll > node->roll)
        node->right = insert(node->right, roll);

    return node;
}

// 1. Finding the student with the smallest roll number (minimum key).
Node* findMin(Node* node) {
    if (node == NULL) return NULL;
    while (node->left != NULL)
        node = node->left;
    return node;
}

// 2. Finding the student with the largest roll number (maximum key).
```

```
Node* findMax(Node* node) {
    if (node == NULL) return NULL;
    while (node->right != NULL)
        node = node->right;
    return node;
}

// 3. Searching for a student using a given roll number.
Node* search(Node* node, int roll) {
    if (node == NULL || node->roll == roll) return node;

    if (roll < node->roll)
        return search(node->left, roll);
    else
        return search(node->right, roll);
}

// 4. Finding the predecessor of a student (the student with the closest smaller roll number).
Node* predecessor(Node* root, int roll) {
    Node* current = search(root, roll);
    if (!current) return NULL;
```

```
    if (current->left != NULL) {
        return findMax(current->left);
    }

    Node* pred = NULL;
    Node* ancestor = root;
    while (ancestor != current) {
        if (current->roll > ancestor->roll) {
            pred = ancestor;
            ancestor = ancestor->right;
        } else {
            ancestor = ancestor->left;
        }
    }
    return pred;
}

// 5. Finding the successor of a student (the student with the closest larger roll number).
Node* successor(Node* root, int roll) {
    Node* current = search(root, roll);
```

```
    if (!current) return NULL;

    if (current->right != NULL) {
        return findMin(current->right);
    }

    Node* succ = NULL;
    Node* ancestor = root;
    while (ancestor != current) {
        if (current->roll < ancestor->roll) {
            succ = ancestor;
            ancestor = ancestor->left;
        } else {
            ancestor = ancestor->right;
        }
    }
    return succ;
}

// 6. Deleting a studentâ€™s record using their roll number
Node* deleteNode(Node* root, int roll) {
```

```
if (root == NULL) return root;

if (roll < root->roll)
    root->left = deleteNode(root->left, roll);

else if (roll > root->roll)
    root->right = deleteNode(root->right, roll);

else {
    if (root->left == NULL) {
        Node* temp = root->right;
        delete root;
        return temp;
    }
    else if (root->right == NULL) {
        Node* temp = root->left;
        delete root;
        return temp;
    }

    Node* temp = findMin(root->right);
    root->roll = temp->roll;
```

```
        root->right = deleteNode(root->right, temp->roll);
    }
    return root;
}

// BST Display- Inorder, Preorder and Postorder.
void inorder(Node* node) {
    if (node == NULL) return;
    inorder(node->left);
    cout << node->roll << " ";
    inorder(node->right);
}

void preorder(Node* node) {
    if (node == NULL) return;
    cout << node->roll << " ";
    preorder(node->left);
    preorder(node->right);
}

void postorder(Node* node) {
    if (node == NULL) return;
```

```
        postorder(node->left);
        postorder(node->right);
        cout << node->roll << " ";
    }
};

int main() {
    BST tree;
    int choice, roll;

    while (true) {
        cout << "\n===== Student Record Management (BST) =====\n";
        cout << "1. Insert student\n";
        cout << "2. Find Minimum roll number\n";
        cout << "3. Find Maximum roll number\n";
        cout << "4. Search student\n";
        cout << "5. Find Predecessor\n";
        cout << "6. Find Successor\n";
        cout << "7. Delete student\n";
        cout << "8. Display Inorder\n";
        cout << "9. Display Preorder\n";
```

```
        cout << "10. Display Postorder\n";
        cout << "11. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter roll number to insert: ";
            cin >> roll;
            tree.root = tree.insert(tree.root, roll);
            break;

        case 2:
            if (tree.findMin(tree.root))
                cout << "Minimum Roll Number: " <<
tree.findMin(tree.root)->roll << endl;
            else
                cout << "Tree is empty!\n";
            break;

        case 3:
            if (tree.findMax(tree.root))
```

```
            cout << "Maximum Roll Number: " <<
tree.findMax(tree.root)->roll << endl;
        else
            cout << "Tree is empty!\n";
        break;

    case 4:
        cout << "Enter roll number to search: ";
        cin >> roll;
        if (tree.search(tree.root, roll))
            cout << "Student Found!\n";
        else
            cout << "Student NOT Found!\n";
        break;

    case 5:
        cout << "Enter roll number to find predecessor: ";
        cin >> roll;
        if (tree.predecessor(tree.root, roll))
            cout << "Predecessor: " <<
tree.predecessor(tree.root, roll)->roll << endl;
        else
```

```
            cout << "No predecessor exists!\n";
        break;

    case 6:
        cout << "Enter roll number to find successor: ";
        cin >> roll;
        if (tree.successor(tree.root, roll))
            cout << "Successor: " << tree.successor(tree.root,
roll)->roll << endl;
        else
            cout << "No successor exists!\n";
        break;

    case 7:
        cout << "Enter roll number to delete: ";
        cin >> roll;
        tree.root = tree.deleteNode(tree.root, roll);
        cout << "Deleted (if present).\n";
        break;

    case 8:
        cout << "Inorder Traversal: ";
```

```
    tree.inorder(tree.root);
    cout << endl;
    break;

case 9:
    cout << "Preorder Traversal: ";
    tree.preorder(tree.root);
    cout << endl;
    break;

case 10:
    cout << "Postorder Traversal: ";
    tree.postorder(tree.root);
    cout << endl;
    break;

case 11:
    return 0;

default:
    cout << "Invalid Choice!\n";
}
```

```
    }
}
```

```
/*
===== Student Record Management (BST) =====
1. Insert student
2. Find Minimum roll number
3. Find Maximum roll number
4. Search student
5. Find Predecessor
6. Find Successor
7. Delete student
8. Display Inorder
9. Display Preorder
10. Display Postorder
11. Exit
Enter your choice: 1
Enter roll number to insert: 56

===== Student Record Management (BST) =====
1. Insert student
```

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 41

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 2

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 71

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 32

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 6

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 38

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 91

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 83

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 1

Enter roll number to insert: 76

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 8

Inorder Traversal: 2 6 32 38 41 56 71 76 83 91

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 9

Preorder Traversal: 56 41 2 32 6 38 71 91 83 76

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 10

Postorder Traversal: 6 38 32 2 41 76 83 91 71 56

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 2

Minimum Roll Number: 2

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 3

Maximum Roll Number: 91

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 4

Enter roll number to search: 41

Student Found!

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 5

Enter roll number to find predecessor: 56

Predecessor: 41

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 38

Invalid Choice!

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 6

Enter roll number to find successor: 38

Successor: 41

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 7

Enter roll number to delete: 91

Deleted (if present).

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 8

Inorder Traversal: 2 6 32 38 41 56 71 76 83

===== Student Record Management (BST) =====

1. Insert student

2. Find Minimum roll number

3. Find Maximum roll number

4. Search student

5. Find Predecessor

6. Find Successor

7. Delete student

8. Display Inorder

9. Display Preorder

10. Display Postorder

11. Exit

Enter your choice: 11

***/**